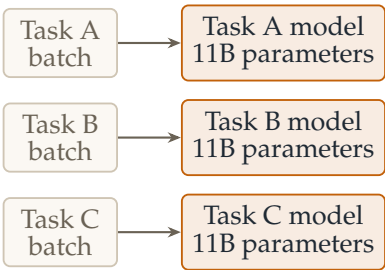
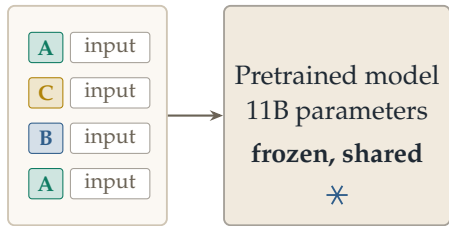


Model tuning



one model copy and
one batch per task

Prompt tuning



mixed-task batch

one shared model; each task adds only its
prompt (20,480 parameters when $m = 5$)

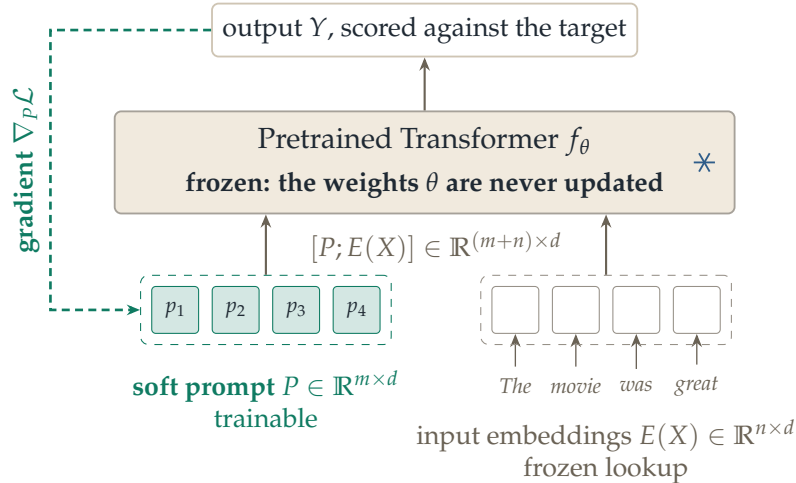
hard prompt: text a person writes

soft prompt: vectors training learns

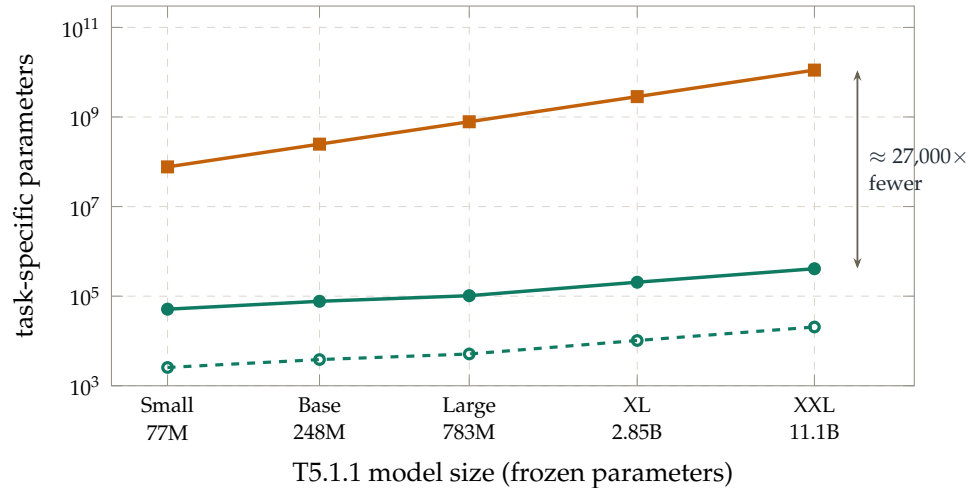
Classify the sentiment
of this movie review as
positive or *negative*:

replaced by

p_1	0.14	-0.72	0.31	0.08
p_2	-0.45	0.19	0.66	-0.27
p_3	0.52	-0.11	0.03	0.94



■ Model tuning (all weights) ● Prompt tuning, $m = 100$ (default) ○ Prompt tuning, $m = 5$



setup

1 Define the task and labeled data

2 Initialize the soft prompt P

3 Freeze the model weights θ

training loop

4 Forward pass on $[P; E(X)]$

5 Cross-entropy loss $\mathcal{L}$

repeat for each mini-batch

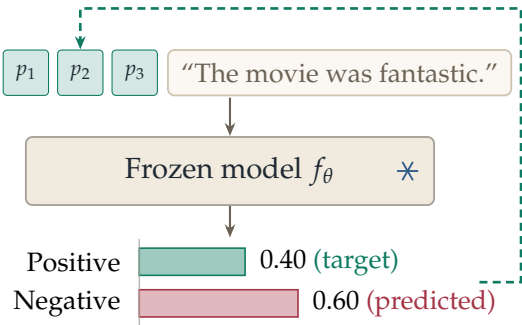
7 Update P only
 $P \leftarrow P - \eta \nabla_P \mathcal{L}$

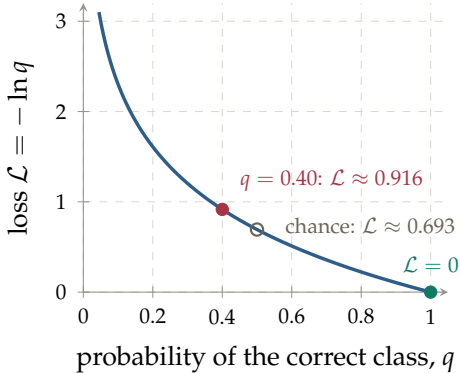
6 Backpropagate to get $\nabla_P \mathcal{L}$

converged

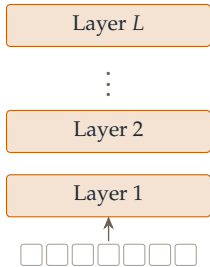
8 Store the task prompt P^* (kilobytes)

$\nabla_P \mathcal{L}$ flows back to P only



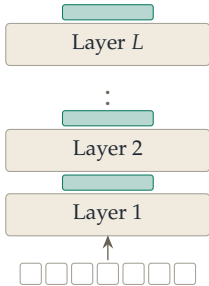


Full fine-tuning



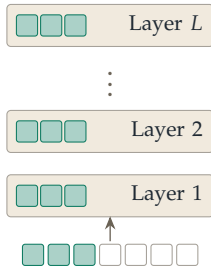
task-specific: **100%**

Adapters



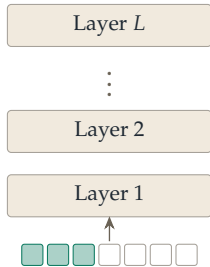
task-specific: **2–4%**

Prefix tuning



task-specific: **0.1–1%**

Prompt tuning



task-specific: **< 0.01%**



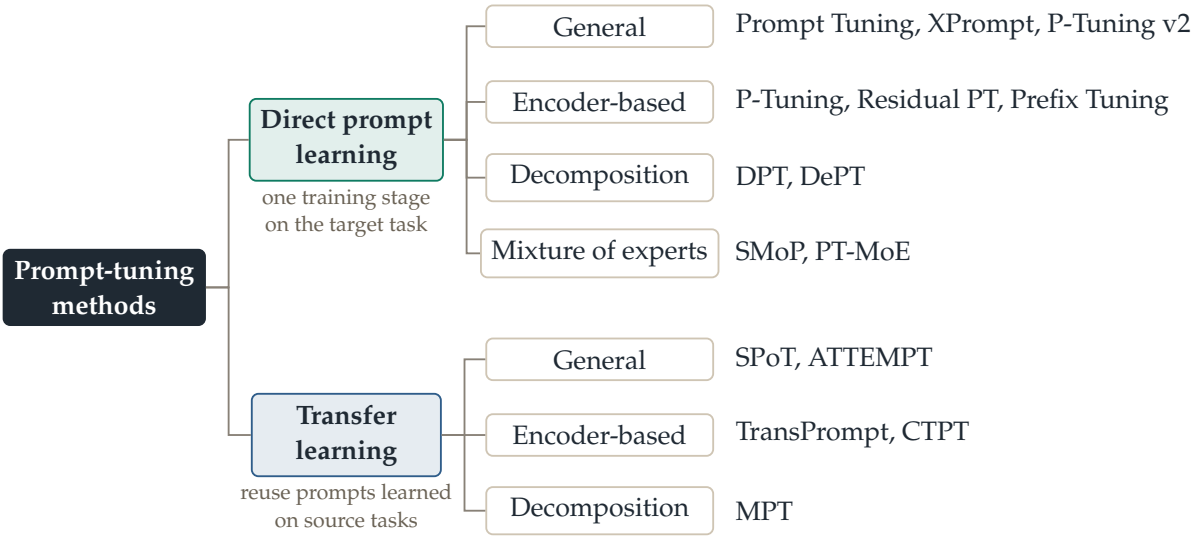
trained (all weights)



trained, task-specific



frozen, shared



**Prompt
engineering**

**Prompt
tuning**

**Full
fine-tuning**



write the prompt
by hand

learn a few vectors,
freeze the model

update every
weight

← faster and cheaper

more control, more cost →

Speed of implementation

Data
requirement

Full fine-tuning

Prompt tuning

Output
quality

Model
modification

Flexibility

